# QDR.AI Complete Handover Guide

Agentic Quality Document Review System -- Architecture, Module Reference & Deployment Guide

| | |
|---|---|
| **Version** | 2.0 |
| **Document Type** | Technical Handover Reference |
| **Pipelines** | HRDR (Hardness Review) & HYDR (Hydrostatic Test) |
| **Tech Stack** | Python / LangGraph / ChromaDB / Ollama / Streamlit / FastAPI |
| **LLM Models** | Qwen 2.5 7B-Instruct & Qwen 2.5VL 7B (Vision) |
| **Embedding Model** | BAAI/bge-m3 (1024-dim dense vectors) |
| **Date** | October 2026 |

# QDR*AI -- Complete Handover Guide

> ***Purpose:*** *This document is the definitive handover reference for anyone taking over the QDR-AI project. It covers how to run the application from scratch, the full system architecture, every module's purpose, and the exact relationships between them.*

## Table of Contents


1. What is QDR-AI?
2. Step-by-Step: Running the Application
3. System Architecture Overview
4. Directory Structure
5. Module-by-Module Breakdown
6. How the Modules Relate to Each Other
7. Two Inspection Pipelines Explained
8. Data Flow: End-to-End Walk-through
9. REST API Reference
10. Environment Variables Reference
11. Troubleshooting


## 1. What is QDR-AI?

QDR-AI is an **industrial-grade, multi-agent AI system** that automates quality assurance and regulatory compliance review of heavy plant engineering supplier documentation. It does two main things:

| Mode | What it Inspects | Techniques Used |
|---|---|---|
| **HRDR** (Hardness Review) | Hardness test reports -- material grades, HBW readings, limits | Spatial OCR, deterministic rule engine, LLM reasoning |
| **HYDR** (Hydrostatic Test) | Pressure test reports -- dial gauges, water chemistry, cross-page consistency | Computer vision (NumPy/OpenCV), two-stage validation, knowledge graph |

The system outputs:

- A **compliance disposition** (Approved / Revise & Resubmit / Rejected)
- A **compliance score** (0-100)
- A list of **specific findings** with page references
- A **downloadable annotated PDF** with highlighted problem areas

## 2. Step-by-Step: Running the Application

### Prerequisites Checklist

Before starting, verify you have:

- [ ] An **NVIDIA GPU** with ? 8 GB VRAM (RTX 3080+ recommended)
- [ ] **CUDA 12.4** and NVIDIA drivers installed
- [ ] **Python 3.10, 3.11, or 3.12**
- [ ] **Git** installed
- [ ] At least **20 GB** free disk space

---

## Step 1 -- Verify GPU and CUDA

```
nvidia-smi
nvcc --version
```

Expected output for `nvidia-smi` should show your GPU name, driver version, and CUDA version.

---

## Step 2 -- Clone the Repository and Create a Virtual Environment

```
# Clone the repository
git clone <your-repo-url>
cd QDR-AI

# Create a Python virtual environment
python3 -m venv .venv

# Activate it -- Linux / macOS:
source .venv/bin/activate

# Activate it -- Windows (PowerShell):
.venv\Scripts\Activate.ps1
```

> ***Why a virtual environment?*** *It isolates QDR-AI's exact package versions from your system Python, preventing version conflicts.*

---

## Step 3 -- Install PyTorch with CUDA Support (CRITICAL -- Do This First)

> **IMPORTANT: PyTorch ? 2.6 is mandatory.** If you install an older version, the BGE-M3 embedding model will fail to load with a `CVE-2025-32434` error, and the entire RAG retrieval system will be unavailable.

```
# Upgrade pip first
pip install --upgrade pip setuptools wheel

# Install PyTorch with CUDA 12.4 support
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu124
```

**Verify PyTorch sees your GPU:**

```
python -c "import torch; print('Torch:', torch.__version__); print('CUDA:',
torch.cuda.is_available()); print('GPU:', torch.cuda.get_device_name(0))"
```

You should see `CUDA: True` and your GPU name.

---

## Step 4 -- Install Application Dependencies

```
pip install -r requirements.txt
```

This installs all remaining libraries: FastAPI, Streamlit, LangChain, LangGraph, ChromaDB, PyMuPDF, OpenCV, SentenceTransformers, etc.

---

## Step 5 -- Install Tesseract OCR Engine

Tesseract is the OCR engine that reads text from scanned PDF pages.

**Ubuntu / Debian:**

```
sudo apt update
sudo apt install -y tesseract-ocr libtesseract-dev tesseract-ocr-eng
which tesseract # Verify path: usually /usr/bin/tesseract
```

**Windows:**

1. Download the installer from UB Mannheim Tesseract
2. Install to the default path: `C:\Program Files\Tesseract-OCR\tesseract.exe`

---

## Step 6 -- Install and Configure Ollama (Local LLM Server)

Ollama runs the Qwen 2.5 language models locally on your GPU.

**Linux:**

```
curl -fsSL https://ollama.com/install.sh | sh
```

**Windows:** Download the installer from https://ollama.com/download

**Configure Ollama for GPU and external access:**

```
export OLLAMA_HOST=0.0.0.0:11434
export OLLAMA_NUM_GPU=1
export CUDA_VISIBLE_DEVICES=0
ollama serve &
```

**Download the required AI models:**

```
ollama pull qwen2.5:7b-instruct # Primary reasoning model (~4.7 GB)
ollama pull qwen2.5:3b # Lightweight fallback (~2 GB)
ollama pull qwen2.5vl:7b # Vision model for gauge images (~4.7 GB)
```

**Verify models are loaded on GPU:**

```
ollama ps
```

---

## Step 7 -- Configure Environment Variables

```
# Copy the example environment file
cp .env.example .env
```

Now edit `.env` to match your setup. Here is the full recommended configuration:

```
# -- Ollama Host Configuration ----------------------------------------
# Point to your GPU server's Ollama instance
OLLAMA_LOCAL_HOST=http://localhost:11434
```

```
OLLAMA_GPU_HOST=http://localhost:11434
OLLAMA_FORCE_LOCAL=true

# -- LLM Models ------------------------------------------------------
DEFAULT_LLM_MODEL=qwen2.5:7b-instruct
REASONING_MODEL=qwen2.5:7b-instruct
OLLAMA_MODEL=qwen2.5:7b-instruct
OLLAMA_LOCAL_MODEL=qwen2.5:3b
OLLAMA_VISION_MODEL=qwen2.5vl:7b

# -- Timeouts (seconds) ----------------------------------------------
OLLAMA_TIMEOUT=300
OLLAMA_ANALYZER_TIMEOUT=600
OLLAMA_ANALYZER_NUM_CTX=8192

# -- Embedding Model -------------------------------------------------
EMBEDDING_MODEL=BAAI/bge-m3
EMBEDDING_DEVICE=cuda # Use "cpu" if no GPU available
EMBEDDING_DIMENSION=1024

# -- OCR Binary Path -------------------------------------------------
# Linux: /usr/bin/tesseract
# Windows: C:\Program Files\Tesseract-OCR\tesseract.exe
TESSERACT_CMD=/usr/bin/tesseract

# -- Backend & Ports -------------------------------------------------
QDR_BACKEND_HOST=0.0.0.0
QDR_BACKEND_PORT=8001
STREAMLIT_PORT=8501

# -- ChromaDB Vector Store -------------------------------------------
CHROMA_PERSIST_DIR=./data/chromadb
```

---

## Step 8 -- Launch the Application

Open **two terminal windows** (both with the virtual environment activated):

**Terminal 1 -- Start the FastAPI Backend:**

```
python -m uvicorn src.api.main:app --host 0.0.0.0 --port 8001
```

You should see:

```
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8001
```

**Terminal 2 -- Start the Streamlit Web UI:**

```
python -m streamlit run app.py --server.port 8501 --server.address 0.0.0.0
```

---

## Step 9 -- Access the Application

| Interface | URL |
|---|---|
| **Web UI (Streamlit)** | `http://localhost:8501` |
| **REST API** | `http://localhost:8001` |
| **Interactive API Docs** | `http://localhost:8001/docs` |

## Step 10 -- Using the Application (First Run)

1. **Open the Web UI** at `http://localhost:8501`
2. **Tab: "Review Guide & Standards"** -- Upload your N/E Review Guide PDF (e.g., HRDR or HYDR review guide)
3. **Tab: "Client Procedures"** -- Upload relevant specification/procedure documents (these get stored in ChromaDB for RAG)
4. **Tab: "Supplier Document"** -- Upload the supplier's submitted quality document (PDF)
5. **Sidebar** -- Fill in metadata (Project Name, Supplier, Document Number, Revision)
6. **Select the Guide Type** from the sidebar: `HRDR` for Hardness Review, `HYDR` for Hydrostatic Test
7. **Click "? Inspect Document"** to run the pipeline
8. **Tab: "Inspection Results"** -- View the report with findings, scores, and download the annotated PDF

**TIP:** The first run will download the BGE-M3 embedding model (~2 GB) from HuggingFace automatically. Subsequent runs use the cached model.

## Production Deployment (Linux Systemd)

For 24/7 uptime, create systemd service files:

`/etc/systemd/system/qdr-api.service`:

```
[Unit]
Description=QDR-AI FastAPI Backend
After=network.target

[Service]
Type=simple
User=ubuntu
WorkingDirectory=/home/ubuntu/QDR-AI
Environment="PATH=/home/ubuntu/QDR-AI/.venv/bin"
ExecStart=/home/ubuntu/QDR-AI/.venv/bin/python -m uvicorn src.api.main:app --host 0.0.0.0
--port 8001 --workers 2
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

`/etc/systemd/system/qdr-ui.service`:

```
[Unit]
Description=QDR-AI Streamlit UI
After=network.target

[Service]
Type=simple
User=ubuntu
WorkingDirectory=/home/ubuntu/QDR-AI
Environment="PATH=/home/ubuntu/QDR-AI/.venv/bin"
ExecStart=/home/ubuntu/QDR-AI/.venv/bin/python -m streamlit run app.py --server.port 8501
--server.address 0.0.0.0 --server.headless true
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable --now qdr-api qdr-ui
sudo systemctl status qdr-api qdr-ui
```

## 3. System Architecture Overview

QDR-AI is composed of **4 major layers** that work together:

```
+-------------------------------------------------------------------------+
| PRESENTATION LAYER |
| +-----------------------+ +-------------------------------+ |
| | Streamlit Web UI | | FastAPI REST Backend | |
| | app.py (Port 8501) | | src/api/main.py (Port 8001) | |
| +-----------------------+ +-------------------------------+ |
+-------------------------------------------------------------------------+
|
+----------------?-----------------+
? ?
+-----------------------------+ +----------------------------------+
| HRDR PIPELINE | | HYDR PIPELINE |
| (Hardness Review) | | (Hydrostatic Test Review) |
| | | |
| orchestrator.py | | hydro_orchestrator.py |
| (LangGraph StateGraph) | | (Sequential multi-stage) |
| | | |
| 1. OCR Extraction | | 1. PDF Ingestion & OCR |
| 2. RAG Context Build | | 2. Review Guide Parsing |
| 3. LLM Finding Analyzer | | 3. Hybrid RAG Retrieval |
| 4. Python Rule Engine | | 4. Knowledge Graph Build |
| 5. Annotation Builder | | 5. Two-Stage Gauge Validation |
| 6. LLM Report Synthesizer | | 6. Water Chemistry Verification |
| | | 7. LLM Executive Summary |
+-----------------------------+ +----------------------------------+
| |
+----------------?-----------------+
?
+-------------------------------------------------------------------------+
| INTELLIGENCE LAYER |
| |
| +----------------------+ +--------------------+ +---------------+ |
| | ChromaDB | | Document | | Ollama LLM | |
| | Vector Store | | Knowledge Graph | | (Qwen 2.5) | |
| | (BGE-M3 1024-dim) | | (NetworkX) | | qwen2.5:7b | |
| +----------------------+ +--------------------+ +---------------+ |
+-------------------------------------------------------------------------+
|
+-------------------------------------------------------------------------+
| INGESTION / PARSING LAYER |
| |
| PyMuPDF (fitz) -> OCR (Tesseract) -> Chunker -> Annotator |
| src/ingestion/parser.py, chunker.py, annotator.py, pipeline.py |
+-------------------------------------------------------------------------+
```

**Key Design Principles:**

- **Deterministic first, LLM second:** All arithmetic (hardness limits, pressure calculations, compliance pass/fail) is done in Python with deterministic rules. LLMs are called only twice: once to *interpret* specifications, once to *narrate* the report.

- **OCR-driven retrieval:** RAG queries are built from what OCR actually finds in the document (material grades, drawing numbers), not generic keywords.
- **No silent fallbacks:** If the embedding model fails to load, or an LLM agent produces no usable output (in `STRICT_LLM_MODE`), the pipeline aborts visibly rather than producing a fake report.

## 4. Directory Structure

```
QDR-AI/
|
+-- app.py <- Streamlit frontend (main UI file)
+-- requirements.txt <- All Python dependencies
+-- .env.example <- Template for environment config
+-- .env <- Your actual config (not committed to git)
+-- README.md <- Deployment guide
+-- DOCUMENTATION.md <- Deep technical specification
|
+-- src/ <- All application source code
| +-- config.py <- Env variable loader + model host selector
| +-- __init__.py
| |
| +-- api/ <- FastAPI REST server
| | +-- main.py <- All API endpoints
| |
| +-- agents/ <- Core AI processing modules
| | +-- orchestrator.py <- HRDR pipeline (LangGraph StateGraph)
| | +-- hydro_orchestrator.py <- HYDR pipeline (sequential stages)
| | +-- state.py <- Shared pipeline state TypedDict
| | +-- ocr_extractor.py <- PDF table extraction via OCR
| | +-- llm_batch_reviewer.py <- LLM Call #1: Finding Analyzer
| | +-- annotation_builder.py <- PDF callout coordinate builder
| | +-- report_synthesizer_agent.py <- LLM Call #2: Report Synthesizer
| | +-- gauge_cv_geometry.py <- OpenCV + NumPy dial gauge reader
| | +-- gauge_vision_agent.py <- Vision LLM gauge fallback
| | +-- two_stage_validator.py<- Stage 1 & 2 pressure validation
| | +-- loop_controller.py <- Retry/loop logic for validation
| | +-- tools.py <- LangChain tools (classify, search, extract)
| | +-- materials.py <- Material grade canonical mapping
| | +-- rule_engine.py <- Deterministic hardness rule checker
| | +-- compliance_auditor_agent.py <- Compliance audit logic
| | +-- classifier.py <- Document type classifier
| | +-- ... (legacy/support files)
| |
| +-- ingestion/ <- Document parsing and storage
| | +-- pipeline.py <- End-to-end ingest orchestrator
| | +-- parser.py <- PDF -> text+tables (PyMuPDF + OCR)
| | +-- chunker.py <- Parent-child text chunker
| | +-- annotator.py <- PDF annotation overlay engine
| |
| +-- knowledge_graph/ <- Semantic document graph
| | +-- document_graph_builder.py <- Cross-page relationship engine
| | +-- graph_builder.py <- Base graph construction
| | +-- hybrid_retriever.py <- Vector + Graph combined retriever
| |
| +-- vectorstore/ <- ChromaDB vector database
| | +-- embeddings.py <- BGE-M3 embedding model loader
| | +-- store.py <- ChromaDB collection manager
| |
| +-- utils/ <- Shared utilities
| +-- logger.py <- Structured pipeline logger
| +-- metrics.py <- Stage timing tracker
| +-- progress.py <- Live UI progress state
```

```
| +-- run_lock.py <- Single-run enforcement lock
| +-- stage_log.py <- Per-run stage audit log
| +-- usage_log.py <- Session usage analytics
| +-- llm_tracer.py <- LLM call trace recorder
|
+-- data/ <- Runtime data (auto-created)
| +-- chromadb/ <- Persistent vector index files
| +-- documents/ <- Uploaded PDF storage
| +-- parsed/ <- OCR caches + JSON trace files
| +-- logs/ <- Pipeline audit logs
| +-- reports/ <- Generated report files
|
+-- scratch_isgec_pages/ <- Sample page images for testing
```

## 5. Module-by-Module Breakdown

### `src/config.py` -- Configuration Hub

**What it does:** Loads all settings from `.env` file and provides them as constants to every other module. Also implements the **Ollama host selection** logic: it tries to reach a remote GPU server first, falls back to localhost if unreachable, and caches the result for 30 seconds to avoid repeated network probes.

**Key exports:**

- `OLLAMA_HOST`, `DEFAULT_LLM_MODEL`, `REASONING_MODEL` -- LLM connection settings
- `TESSERACT_CMD` -- Path to Tesseract binary
- `EMBEDDING_MODEL_NAME`, `EMBED_TOP_K` -- Vector search settings
- `AGENT_MODE` -- `"langgraph"` (AI pipeline) or `"rule_based"` (legacy)
- `STRICT_LLM_MODE` -- If `True`, aborts rather than using fallback values
- `get_ollama_host()` -- Dynamic host resolver with caching

### `app.py` -- Streamlit Web Frontend

**What it does:** The entire web user interface. A single-file Streamlit application that provides:

- **4 Tabs:** Review Guide upload, Client Procedures upload, Supplier Document upload, Inspection Results display
- **Sidebar:** Discipline selector, review mode (Initial/Re-submittal), guide type (HRDR/HYDR), all metadata fields
- **Run Button:** Triggers the pipeline in a background thread while showing live progress
- **Results rendering:** Findings cards, compliance scores, annotated PDF download
- **Auto-metadata extraction:** Parses the supplier name, document number, revision, and MAWP from the uploaded document's text using regex patterns

**Key function:** `_run_pipeline_in_background()` -- Runs in a `threading.Thread` so the UI stays responsive during processing. It automatically routes to `run_inspection_pipeline()` (HRDR) or `run_hydrostatic_inspection_pipeline()` (HYDR) based on filename and guide type.

### `src/api/main.py` -- FastAPI Backend

**What it does:** A REST API server that exposes the same inspection pipeline over HTTP. Enables programmatic integration without using the web UI.

**Endpoints:**

| Method | Path | Purpose |
|---|---|---|
| GET | / | System status |
| GET | /health | Model availability check |
| POST | /inspect | Run HRDR inspection pipeline |
| POST | /inspect/annotated-pdf | Run inspection + return annotated PDF as download |
| GET | /search | Query ChromaDB vector store |
| GET | /logs | Tail the pipeline log file |

**Startup action:** Warms up the BGE-M3 embedding model at boot time to avoid a 17-second cold-load on the first actual inspection.

---

## src/agents/state.py -- Shared Pipeline State

**What it does:** Defines ReviewState, a TypedDict that is the single shared data structure passed between every node in the HRDR LangGraph pipeline. Every agent reads from and writes to this state object.

**Key fields in ReviewState:**

- submittal_text, submittal_filename, submittal_filepath -- Input document
- page_ocr_results -- Structured OCR output (list of PageOCRResult)
- detected_materials -- Material grades found by OCR
- retrieved_criteria -- Spec clauses from ChromaDB
- hardness_rules_schema -- Limits established by LLM Analyzer
- raw_rule_failures -- Every row that fails the rule engine
- findings -- Final finding list for the report
- disposition -- "APPROVED" / "REVISE AND RESUBMIT" / "REJECTED"
- score -- Numeric compliance score (0-100)
- summary -- Executive narrative (LLM-generated)

---

## src/agents/orchestrator.py -- HRDR LangGraph Pipeline

**What it does:** Assembles the HRDR inspection pipeline as a **LangGraph StateGraph** -- a directed acyclic graph of processing nodes. Each node is a Python function that receives the current ReviewState and returns a dict of updates to merge back into it.

**Pipeline nodes (in order):**

```
ocr_extract -> build_rag_context -> llm_batch_review -> build_annotations -> synthesize_report
-> END
```

| Node | Type | What It Does |
|---|---|---|
| ocr_extract | Python | Runs OCR on every page, extracts tables and hardness readings |
| build_rag_context | Python + ChromaDB | Builds material-aware RAG queries from OCR results, retrieves spec clauses |
| llm_batch_review | **LLM Call #1** | Reads retrieved specs, establishes hardness limits, groups non-conformances |

| Node | Type | What It Does |
|---|---|---|
| `build_annotations` | Python | Converts findings to PDF pixel coordinate bounding boxes |
| `synthesize_report` | **LLM Call #2** | Writes the executive narrative (scoring is deterministic) |

**Fallback:** If `AGENT_MODE=rule_based`, the entire LangGraph pipeline is bypassed and the legacy `qa_inspector_agent_legacy.py` runs instead.

---

## `src/agents/hydro_orchestrator.py` -- HYDR Pipeline

**What it does:** Implements the Hydrostatic Test inspection as a sequential multi-stage pipeline (not LangGraph). More complex than HRDR because it needs to handle:

- Multi-page cross-referencing (Sheet 1 table data vs Sheet 2 pressure chart photos)
- Physical dial gauge image analysis via computer vision
- Water chemistry compliance (chlorides, pH, hardness)
- Two-stage validation (Stage 1: OCR vs visual consistency, Stage 2: code compliance)

**Pipeline stages (in order):**

1. **Parse PDF** -- `parse_pdf()` from ingestion module
2. **Parse Review Guide** -- Extract active compliance criteria
3. **Hybrid RAG Retrieval** -- Query ChromaDB + Knowledge Graph
4. **Document Knowledge Graph** -- Build cross-page entity relationship graph
5. **Two-Stage Gauge Validation** -- For each test step (0%, 50%, 100%, MAWP, Water Temp, Metal Temp)
6. **Water Chemistry Evaluation** -- Chlorides ? 50 ppm, pH 7-10, hardness check
7. **LLM Executive Summary** -- Single Ollama call to generate the narrative

---

## `src/agents/ocr_extractor.py` -- OCR Table Extractor

**What it does:** The first and most critical agent for HRDR. Uses a combination of:

- **PyMuPDF** (`fitz`) for native text extraction from digital PDFs
- **pdfplumber** for table structure detection
- **Tesseract** via `pytesseract` for scanned/image-based pages

Extracts structured data from hardness test tables:

- Row-by-row readings (spool number, weld ID, material grade, HBW values)
- Page-level metadata (drawing number, test block info, acceptance ranges)

Returns a list of `PageOCRResult` objects, one per page.

---

## `src/agents/llm_batch_reviewer.py` -- LLM Finding Analyzer (Call #1)

**What it does:** The first and most important LLM call in the HRDR pipeline. It receives:

- The summary of all OCR-extracted readings (materials, HBW values, ranges found)
- The specification clauses retrieved from ChromaDB

It returns:

- The exact hardness limits (HBW min/max) for each material grade, **with a citation** to the spec clause that establishes them
- A grouping of non-conforming rows by root cause

**NOTE:** This call does NOT evaluate individual rows -- that's done by the deterministic Python rule engine. Its job is only to **interpret the specification** and establish limits that the rule engine will apply universally.

---

### `src/agents/annotation_builder.py` -- PDF Annotation Builder

**What it does:** Converts the abstract list of findings (page index, finding type, comment) into precise PDF pixel coordinate bounding boxes that the annotator can draw on the PDF. Maps findings to the correct page and approximate visual region based on the source table row's OCR coordinate data.

---

### `src/agents/report_synthesizer_agent.py` -- Report Synthesizer (Call #2)

**What it does:** The second LLM call. Receives the complete findings summary and writes the executive narrative for the final report. Importantly, it does **not** determine the disposition or score -- those are calculated deterministically in Python based on the count and severity of findings. The LLM only writes the human-readable explanation.

---

### `src/agents/gauge_cv_geometry.py` -- Computer Vision Gauge Reader

**What it does:** The most technically sophisticated module. Pure OpenCV + NumPy computer vision that reads physical pressure gauge dial images:

1. **Instrument Classification** -- Distinguishes analog pressure gauges, digital pressure meters, IR thermometers, serial number plates
2. **Gauge Face Detection** -- Finds the circular gauge face using Hough Circle Transform
3. **Needle Detection** -- Uses Canny edge detection + Hough Line Transform to find the needle
4. **Polar Angle Measurement** -- Calculates the exact angle ? from the gauge center to the needle tip
5. **Pressure Calculation** -- Maps ? to PSI using the dial's known range (0-300, 0-500, or 0-2000 PSI)
6. **LCD OCR** -- Reads digital temperature displays using 7-segment digit recognition

Uses **zero static placeholders** -- every reading is derived from actual pixel analysis.

---

### `src/agents/two_stage_validator.py` -- Two-Stage Validator

**What it does:** Implements the two-stage quality validation framework for hydrostatic tests:

- **Stage 1 (Consistency Check):** Does the computer vision gauge reading agree with what the QA inspector manually recorded in the table? (within ?2 PSIG tolerance)
- **Stage 2 (Specification Compliance):** Does the confirmed reading meet the code requirement (e.g., ? 1.5? MAWP per ASME Section I)?

Produces a `GaugeValidationReport` for each measurement, with possible dispositions: `ACCEPTED`, `EXTRACTION_MISMATCH`, `NON_CONFORMANCE`, `HUMAN_REVIEW_REQUIRED`.

---

### `src/agents/loop_controller.py` -- Validation Loop Controller

**What it does:** Wraps the `TwoStageValidator` with retry logic. If a Stage 1 or Stage 2 check fails, it can trigger a re-retrieval from the knowledge graph/RAG to see if additional specification context changes the outcome. Configured with `max_retries=3`.

---

### `src/agents/tools.py` -- LangChain Tools

**What it does:** Defines the three LangChain `Tool` objects used within the HRDR pipeline's RAG context build step:

- `classify_document` -- Identifies document type (hardness report, hydro test, etc.) and primary material grade from text
- `extract_requirements` -- Converts the Review Guide checklist into a structured list of requirement strings
- `search_specs` -- Executes a ChromaDB similarity search and returns ranked spec clause chunks

---

### `src/agents/materials.py` -- Material Grade Canonical Mapping

**What it does:** Provides the `canonical_grade()` and `retrieval_terms()` functions. Maps every possible way a material might appear in a document (grade name, P-Number, ASTM designation) to a canonical internal identifier.

**Example mappings:**

- `"P91"`, `"F91"`, `"T91"`, `"9Cr"`, `"P5A"` -> all canonicalize to `"P91"`
- `"P22"`, `"F22"`, `"2.25Cr"`, `"P4"` -> all canonicalize to `"P22"`
- `retrieval_terms("P91")` -> returns `"P91 F91 T91 9Cr P5A alloy steel"` (for richer RAG queries)

---

### `src/ingestion/parser.py` -- PDF Parser

**What it does:** The universal PDF text extraction engine. Supports:

- **Digital PDFs** -- Uses PyMuPDF native text extraction (fast, accurate)
- **Scanned PDFs** -- Rasterizes pages at 300 DPI and runs Tesseract OCR
- **Mixed PDFs** -- Page-by-page detection: uses digital text if present, falls back to OCR

Also provides:

- `extract_checklist_items()` -- Parses checkbox-style review guides (detects [x] vs [ ])
- `parse_review_guide()` -- Handles .docx, .docm, .pdf, and raw text review guide formats
- **OCR disk cache** -- Hashes each page's image and caches Tesseract results. Identical pages load in < 10ms instead of re-running OCR.

---

### `src/ingestion/chunker.py` -- Document Chunker

**What it does:** Splits parsed documents into **parent-child chunk pairs** for ChromaDB storage:

- **Child chunks** (~400 tokens): Small, precise chunks used for similarity search
- **Parent chunks** (~1500 tokens): Larger surrounding context returned when a child chunk matches

This "parent-child retrieval" pattern provides both precision (small chunks for accurate matching) and context (larger chunks for the LLM to read).

**Settings (from config):**

- `CHUNK_SIZE_CHILD = 400` tokens
- `CHUNK_SIZE_PARENT = 1500` tokens
- `CHUNK_OVERLAP = 50` tokens

---

### `src/ingestion/annotator.py` -- PDF Annotation Engine

**What it does:** Draws visual annotations directly on the supplier's PDF:

- **Red/orange callout boxes** around non-conforming areas
- **Color-coded severity markers** (red = HOLD/major, orange = major, amber = minor, green = approved)
- **A summary banner** at the top of the first page with the overall disposition and score

- **Multi-threaded rendering** -- Annotates all pages in parallel for speed (< 1.8 seconds for a typical document)

Uses **Pillow** (PIL) for pixel-precise drawing over PyMuPDF rasterized page images.

---

### `src/knowledge_graph/document_graph_builder.py` -- Document Knowledge Graph

**What it does:** Builds a **NetworkX directed graph** that represents the semantic relationships between different parts of a hydrostatic test submittal:

- Links **Sheet 1 manual table records** -> **Sheet 2 pressure chart pages** (same test, different representations)
- Links **hydro reports** -> **gauge calibration certificates** (pages 11-20 typically)
- Links **all reports** -> **specification rules** retrieved from ChromaDB/Knowledge Graph
- Evaluates **cross-page consistency**: does the chart show the same pressure the table records?

The graph enables the HYDR pipeline to answer: "Are Sheet 1's reported values consistent with the visual evidence on Sheet 2?"

---

### `src/knowledge_graph/hybrid_retriever.py` -- Hybrid Retriever

**What it does:** Combines two retrieval strategies:

1. **Vector Retrieval** -- ChromaDB similarity search using BGE-M3 embeddings (semantic matching)
2. **Knowledge Graph Rules** -- Pre-built structured rules for hydrostatic tests (ASME pressure multipliers, water chemistry limits, temperature requirements)

Returns a unified result with both `vector_results` (spec clause text) and `graph_rules` (structured compliance parameters).

---

### `src/vectorstore/embeddings.py` -- BGE-M3 Embedding Loader

**What it does:** Loads the `BAAI/bge-m3` sentence embedding model as a singleton. This model converts text into 1024-dimensional vectors that are stored in ChromaDB. Uses `HuggingFaceEmbeddings` from LangChain with:

- `normalize_embeddings=True` -- Ensures cosine similarity works correctly
- GPU acceleration when `EMBEDDING_DEVICE=cuda`

> **WARNING: Fails hard if PyTorch < 2.6.** The model weights are in `.bin` format, and PyTorch ? 2.6 is required to load them safely (CVE-2025-32434). There is no fallback -- the pipeline aborts with `EmbeddingModelError` rather than using random vectors.

---

### `src/vectorstore/store.py` -- ChromaDB Vector Store Manager

**What it does:** Manages ChromaDB collections (one per guide code, e.g., `"references_procedures"`). Handles:

- `ingest_chunks()` -- Embeds and stores child/parent chunk pairs
- `search()` -- Similarity search with score thresholds, returns parent chunks for full context
- Collection persistence to `./data/chromadb/`

---

### `src/utils/` -- Utility Modules

| File | Purpose |
|---|---|
| `logger.py` | Structured Rich logger with file output to `data/logs/pipeline.log` |

| File | Purpose |
|---|---|
| `metrics.py` | `StageTracker` context manager that times each pipeline stage |
| `progress.py` | Thread-safe progress state (stage name + percentage) read by the UI |
| `run_lock.py` | Prevents two pipeline runs from executing simultaneously |
| `stage_log.py` | Per-run append-only log of which stages executed and how long each took |
| `usage_log.py` | Session analytics: tracks which documents were inspected, outcomes, errors |
| `llm_tracer.py` | Records every LLM call (prompt, response, latency) for debugging |

## 6. How the Modules Relate to Each Other

```
USER
|
?
app.py (Streamlit UI)
|
+-- Uploads PDF -> ingestion/parser.py -> returns text + tables
|
+-- Uploads Review Guide -> ingestion/pipeline.py
| +-- parser.py -> chunker.py -> vectorstore/store.py (stored in ChromaDB)
|
+-- User clicks "Inspect" -> routes to one of two pipelines:
|
| HRDR path:
| agents/orchestrator.py (LangGraph StateGraph)
| +-- Node 1: agents/ocr_extractor.py
| | +-- Uses: ingestion/parser.py (PyMuPDF + Tesseract)
| | +-- Returns: page_ocr_results (table rows with material + HBW)
| |
| +-- Node 2: build_rag_context (inline in orchestrator.py)
| | +-- Uses: agents/tools.py (classify_document, extract_requirements)
| | +-- Uses: agents/materials.py (canonical_grade, retrieval_terms)
| | +-- Uses: vectorstore/store.py -> embeddings.py (BGE-M3 search)
| |
| +-- Node 3: agents/llm_batch_reviewer.py <- LLM Call #1
| | +-- Uses: config.py (get_ollama_host) -> Ollama API
| |
| +-- Node 4: agents/annotation_builder.py
| | +-- Returns: findings with PDF coordinate bounding boxes
| |
| +-- Node 5: agents/report_synthesizer_agent.py <- LLM Call #2
| +-- Uses: config.py (get_ollama_host) -> Ollama API
|
| HYDR path:
| agents/hydro_orchestrator.py (sequential)
| +-- Stage 1: ingestion/parser.py
| +-- Stage 2: ingestion/parser.py (parse_review_guide)
| +-- Stage 3: knowledge_graph/hybrid_retriever.py
| | +-- Uses: vectorstore/store.py (ChromaDB search)
| | +-- Returns: vector_results + graph_rules
| |
```

```
| +-- Stage 4: knowledge_graph/document_graph_builder.py (NetworkX graph)
| |
| +-- Stage 5: agents/two_stage_validator.py + loop_controller.py
| | +-- Uses: agents/gauge_cv_geometry.py (OpenCV/NumPy dial reading)
| | +-- Uses: agents/gauge_vision_agent.py (Vision LLM fallback)
| |
| +-- Stage 6: _evaluate_water_chemistry() (inline, rule-based)
| |
| +-- Stage 7: _synthesize_hydro_narrative_with_llm() <- LLM Call
| +-- Uses: config.py -> Ollama API
|
+-- Post-pipeline: ingestion/annotator.py
+-- Draws annotations on PDF -> returns annotated PDF bytes for download
```

## 7. Two Inspection Pipelines Explained

### HRDR Pipeline -- "Did the hardness readings pass?"

**Goal:** Verify that every weld, base metal, and HAZ hardness reading in a hardness test report is within the limits set by the project specification for the stated material grade.

**The 2 LLM calls explained:**

1. **LLM Call #1 (Finding Analyzer):** Given the retrieved spec clauses and a summary of what OCR found in the document, establish the exact HBW limits for each detected material grade (e.g., "P91 weld: max 265 HBW per Spec 700-006 Table 8, Clause 4.2.1"). Group readings that exceed limits by root cause.
2. **LLM Call #2 (Report Synthesizer):** Given all the findings, write a professional QA narrative explaining what was found, why it matters, and what action is required.

**Why compliance arithmetic stays in Python (not LLM):** A typical hardness report has ~9,000 readings. Sending 9,000 rows to a 4096-token LLM context is impossible. The LLM establishes the limits; Python applies those limits to every row with exact arithmetic -- 100% reproducible.

### HYDR Pipeline -- "Did the hydrostatic test pass?"

**Goal:** Verify that a hydrostatic pressure test was conducted correctly by cross-referencing:

- The pressures the QA inspector manually recorded in the test table (Sheet 1)
- The actual gauge dial readings visible in the test photos (Sheet 2)
- ASME code compliance (test pressure ? 1.5? MAWP)
- Water chemistry (chlorides, pH, temperature)

**Two-Stage Validation framework:**

- **Stage 1:** Does CV gauge reading ? manually recorded value? (consistency check, tolerance ?2 PSIG)
- **Stage 2:** Does the confirmed reading ? specification minimum? (code compliance check)

A pass on both stages = `ACCEPTED`. Fail on Stage 1 = `EXTRACTION_MISMATCH` (data recording error). Fail on Stage 2 only = `NON_CONFORMANCE` (genuine pressure deficiency).

## 8. Data Flow: End-to-End Walk-through

**Scenario: A user uploads an HRDR submittal and clicks "Inspect Document"**

```
1. User uploads PDF in Tab 3 (Supplier Document)
```

```
+-- app.py saves bytes to session_state and calls parse_pdf()
+-- ingestion/parser.py returns {full_text, pages[]}

2. User clicks "? Inspect Document"
+-- app.py spawns a background thread
+-- _run_pipeline_in_background() calls run_inspection_pipeline()
+-- agents/orchestrator.py builds LangGraph graph (once, cached)

3. Node: ocr_extract
+-- agents/ocr_extractor.py
+-- For each page: try PyMuPDF digital text; if scanned -> Tesseract OCR
+-- pdfplumber extracts table structure
+-- Returns: page_ocr_results[] with table_rows[] per page
Each row has: material_hint, hbw_values[], spool_no, etc.

4. Node: build_rag_context
+-- agents/orchestrator.py (inline)
+-- Runs agents/tools.classify_document() on full_text
| -> doc_type="hardness_report", primary_grade="P91"
+-- Extracts detected_materials from ocr_results (canonicalized via materials.py)
| -> ["P91", "P22", "CS"]
+-- Builds RAG queries:
| Query 1: "P91 F91 T91 9Cr P5A ... hardness acceptance criteria HBW..."
| Query 2: "table material grade Brinell HB Max permissible..."
| Query 3: "hardness acceptance range 120 to 265 HBW permissible..."
+-- vectorstore/store.py.search() -> BGE-M3 embeddings -> ChromaDB
-> retrieved_criteria[] = 15 spec clause chunks

5. Node: llm_batch_review (LLM Call #1)
+-- agents/llm_batch_reviewer.py
+-- Sends to Ollama: reading summary + retrieved spec clauses
+-- Ollama (qwen2.5:7b-instruct) responds with:
| hardness_rules_schema = {
| "P91": {"base_metal": {"max": 265}, "weld": {"max": 265}, "haz": {"max": 265}},
| "P22": {"base_metal": {"max": 241}, ...},
| }
+-- Python rule engine then applies these limits to EVERY row in ocr_results
-> raw_rule_failures[] = rows where HBW > limit

6. Node: build_annotations
+-- agents/annotation_builder.py
+-- Maps each failure to its page_idx and approximate PDF coordinates
+-- grouped_findings[] = findings with bounding boxes

7. Node: synthesize_report (LLM Call #2)
+-- agents/report_synthesizer_agent.py
+-- Deterministic: score = 100 ? (passing rows / total rows)
+-- Deterministic: disposition = "APPROVED" if score ? 95 else "REVISE AND RESUBMIT"
+-- LLM writes: summary narrative paragraph
-> final report dict returned to app.py

8. Post-pipeline in app.py
+-- ingestion/annotator.py
+-- Opens original PDF with PyMuPDF
+-- For each finding: draws colored callout box at bounding box coordinates
+-- Draws summary banner on page 1
+-- Returns: annotated_pdf_bytes (downloadable)

9. UI renders results in Tab 4
+-- Findings cards, compliance score gauge, disposition badge
+-- "? Download Annotated PDF" button
```

## 9. REST API Reference

### GET /health

```
{
"status": "healthy",
"llm_model": "qwen2.5:7b-instruct",
"embedding_model": "BAAI/bge-m3",
"vector_store": "ChromaDB",
"strict_llm_mode": false
}
```

### POST /inspect

```
// Request body:
{
"submittal_text": "Full text extracted from supplier PDF...",
"submittal_filename": "HRDR_Report_Rev0.pdf",
"submittal_filepath": "/path/to/file.pdf",
"guide_code": "references_procedures",
"supplier": "ISGEC Heavy Engineering",
"doc_no": "QDR-2024-001",
"revision": "0",
"reviewer": "Senior QA Inspector",
"gen_params_input": {
"proj_name": "Plant QA Project",
"po_num": "TJ/22-23/096"
}
}

// Response: Full ReviewState dict with findings, score, disposition, summary
```

### GET /search?query=P91+hardness+limit&guide_code=references_procedures&k=5

```
{
"query": "P91 hardness limit",
"guide_code": "references_procedures",
"count": 5,
"results": [
{"content": "Alloy 91 (P91/F91) maximum permissible hardness...", "score": 0.92},
...
]
}
```

## 10. Environment Variables Reference

| Variable | Default | Description |
|---|---|---|
| OLLAMA_GPU_HOST | http://172.16.116.228:11435 | Primary GPU server for Ollama |
| OLLAMA_LOCAL_HOST | http://localhost:11434 | Local Ollama fallback |
| OLLAMA_FORCE_LOCAL | false | Skip primary probe, always use localhost |

| Variable | Default | Description |
|---|---|---|
| DEFAULT_LLM_MODEL | qwen2.5:7b-instruct | Primary text reasoning model |
| OLLAMA_LOCAL_MODEL | qwen2.5:3b | Fallback model for localhost |
| OLLAMA_VISION_MODEL | qwen2.5vl:7b | Vision model for gauge images |
| OLLAMA_TIMEOUT | 300 | Seconds for standard LLM calls |
| OLLAMA_ANALYZER_TIMEOUT | 600 | Seconds for the large analyzer call |
| OLLAMA_ANALYZER_NUM_CTX | 8192 | Context window tokens for analyzer |
| EMBEDDING_MODEL | BAAI/bge-m3 | HuggingFace embedding model |
| EMBEDDING_DEVICE | cpu | cuda for GPU acceleration |
| TESSERACT_CMD | C:\Program Files\Tesseract-OCR\tesseract.exe | Tesseract binary path |
| QDR_BACKEND_PORT | 8001 | FastAPI server port |
| CHROMA_PERSIST_DIR | ./data/chromadb | ChromaDB data directory |
| AGENT_MODE | langgraph | langgraph or rule_based |
| STRICT_LLM_MODE | false | Abort instead of using fallback values |
| MAX_LLM_CANDIDATES_COMPLIANCE | 2 | Max rows sent to LLM per document |

## 11. Troubleshooting

### EmbeddingModelError: Failed to load embedding model BAAI/bge-m3

**Cause:** PyTorch < 2.6 is installed.

**Fix:** Reinstall PyTorch with the cu124 wheel (Step 3 above).

### TesseractNotFoundError

**Cause:** Tesseract binary not found at the configured path.

**Fix:**

```
# Linux: find the path
which tesseract
# Then set in .env:
TESSERACT_CMD=/usr/bin/tesseract
```

### Connection refused when calling Ollama

**Cause:** Ollama is not running, or bound to the wrong interface.

**Fix:**

```
# Check if Ollama is running:
curl http://localhost:11434/api/tags
# If not, start it:
ollama serve
```

### Port 8501 or 8001 already in use

**Fix (Linux):**

```
fuser -k 8501/tcp
fuser -k 8001/tcp
```

**Fix (Windows):**

```
netstat -ano | findstr :8501
taskkill /PID <PID> /F
```

### UI shows "Backend Unreachable"

**Cause:** The Streamlit sidebar's "BACKEND URL" field doesn't match where uvicorn is listening.

**Fix:** Ensure `QDR_BACKEND_PORT` in `.env` matches the port you started uvicorn on, and verify uvicorn is actually running.

### LLM calls timing out / very slow

**Cause:** Ollama is running on CPU only (no GPU offload).

**Fixes:**

- Set `OLLAMA_NUM_GPU=1` when starting Ollama
- Reduce model size: set `DEFAULT_LLM_MODEL=qwen2.5:3b` in `.env`
- Reduce context: set `OLLAMA_ANALYZER_NUM_CTX=4096`
- Increase timeout: set `OLLAMA_ANALYZER_TIMEOUT=900`

### `CUDA out of memory`

**Cause:** GPU VRAM exhausted by model + embedding model + other processes.

**Fixes:**

- Use `OLLAMA_NUM_GPU=1` to limit GPU layers
- Switch to `qwen2.5:3b` (smaller model)
- Close other GPU processes
- Set `EMBEDDING_DEVICE=cpu` to offload embeddings to RAM

---

*Document generated: October 2026 | QDR-AI v2.0*